

DengAI: Predicting Disease Spread



Mine Alsan, Viktoria Leuschner, Lucas Rodrigues 🤪

Data Overview



Dengue Dataset:

- 24 columns: Temperature 🌡️, Precipitation 🌧️, Vegetation 🌿
- Column types:
 - Majority float,
 - Year, week int,
 - start_week_date, city object

Decisions:

- Drop columns:
 - start_week_date
- One-Hot Encodings:
 - city

Data Anomalie Checks

Dengue Dataset:

- Missing Values:
 - All except city, year, week
 - City 'sj':
 - station_avg_temp_c, station_diur_temp_rng_c -> 7%
 - station_precip_mm, station_max_temp_c -> 3%
 - station_min_temp_c -> 1.5%
 - remainings -> less than 1 %
 - City 'iq':
 - ndvi_ne -> 20%
 - ndvi_nw -> 5 %
 - ndvi_sw, ndvi_se -> 2%
 - remanings -> less than 1 %

Decisions:

- Mean imputer
- Interpolation imputer

Data Anomalie Checks

Dengue Dataset:

- Range of values:
 - No negative values except for indices
- Histograms with Boxplots:
 - Majority of features have some outliers
- Correlation Heatmap:
 - 2 correlations > 0.9

Decisions:

- Try model with and without outliers
- Drop one of the columns with high correlation:
 - reanalysis_sat_precip_amt_mm, reanalysis_dew_point_temp_k

Target Analysis

Dengue Dataset:

- Histograms with Boxplots:
 - City 'sj':
 - Right skewed distribution
 - Outliers from 100 to 400 cases
 - City 'iq':
 - Right skewed distribution, more extreme peak of 0 cases
 - Outliers from 20 to 120 cases
- Correlation with features
 - City 'sj':
 - Highest with week = 0.21
 - City 'iq':
 - Highest with reanalysis_specific_humidity_g_per_kg = 0.24
 - Extremely low with week = -0.01!

Decisions:

- Log-transform the target
- Try two models for each city
- Add cyclical transformer to account for cyclicity of weeks

Evaluation

Cross-Validation

- Multiple Models
 - LinearRegression, Ridge, Lasso, ElasticNet, BayesianRidge, SGDRegressor, KNeighborsRegressor, SVR, KernelRidge, DecisionTreeRegressor, RandomForestRegressor, ExtraTreesRegressor, GradientBoostingRegressor, GaussianProcessRegressor, MLPRegressor, XGBRegressor, LGBMRegressor
- Random vs Time-aware CV
- Scoring:
 - R2 validation scores almost always negative
 - MAE used for scoring on drivendata.org

Decisions: ✖

- Use Train MAE to pick the best model

One Model vs Two Models

Two Models:

- Best Model
 - ExtraTreeRegressor for both cities
 - Outliers removed with z-score
 - Score: 26.9087

One Model:

- Baseline Model & Best Model
 - RandomForestRegressor
 - No outlier removal
 - Score 26.7139

Decisions:

- Use one model approach!

Features Engineering: Failed

Custom Transformers:

- LaggedTarget: Case counts from previous weeks

Not Available as we do not have Y_test!!

Feature Engineering: Failed

Custom Transformers:

- InterpolationImputer: Use linear interpolation to fill missing values
- OutlierRemover: Remove outlier with z-score or IQR methods
- TempHumidityLagTransformer: create lagged temp x humidity
 - `df['temp_k'] = (df['tmin'] + df['tmax']) / 2`
 - `df['temp_humidity_lag'] = df['temp_k'].shift(lag) * df['rel_humidity'].shift(lag)`
- LaggedNDVIRainInteractionTransformer: mean lagged NDVI x rainfall
 - `df['ndvi_mean'] = df[['ndvi_ne', 'ndvi_nw', 'ndvi_se', 'ndvi_sw']].mean(axis=1)`
 - `df['rain_ndvi_lag3'] = (df['ndvi_mean'] * df['precipitation_amt_mm']).shift(lag)`

Feature Engineering: Failed con't

Custom Transformers:

- **LaggedHeatHumidityStressIndexTransformer:** lagged stress index
 - `df['stress_idx'] = (df['max_temp'] - df['min_temp']) * df['rel_humidity']`
 - `df['stress_idx_lag'] = df['stress_idx'].shift(lag)`
- **MultiFeatureLagTransformer:** Compute lagged versions for multiple feature
 - `df[f'precip_lag{lag}'] = df['precipitation_amt_mm'].shift(lag)`
 - `df[f'ndvi_lag{lag}'] = df['ndvi_mean'].shift(lag)`
- **RollingRainSumTransformer:** Rolling precipitation sum
 - `df['rolling_rain_4w'] = df['precipitation_amt_mm'].rolling(4).sum()`
- **WeatherNDVIANomalyTransformer:** Compute weather anomalies
 - `clim = df['ndvi_mean'].transform('mean')`
 - `df['ndvi_anomaly'] = df['ndvi_mean'] - clim`

Feature Engineering: Succeeded

Custom Transformers:

- **CyclicTransformer**: Encodes a cyclical feature (week of year) into sine and cosine components to preserve periodicity.
 - `df["week_sin"], df["week_cos"] = np.sin(2*np.pi*df["week"]/52), np.cos(2*np.pi*df["week"]/52)`
- **RollingAverageTransformer**: Computes the rolling mean of specified columns over a fixed window separately for each city.
 - `df[f"{col}_rolling_avg_{window}"] = df[col].rolling(window).mean()`

Reality: Lots of experimentation!

Final Model:

- RandomForestRegressor 
- No outlier removal
- Dropped more columns:
 - week_start_date, reanalysis_sat_precip_amt_mm, reanalysis_dew_point_temp_k, ndvi_nw, ndvi_se, reanalysis_air_temp_k, reanalysis_tdtr_k
- Rolling averages for all remaining numerical columns:
 - Window size = 3
- Score: 25.48
- Predictions: 
 - Overestimates low case counts in train data
 - Underestimates pikes in train data

